

Zachary D'Urso

Preliminary Results

```
# import necessary libraries for preliminary results
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
from google.colab import files
```

```
# upload file to use
uploaded = files.upload()
data = pd.read_csv('project_data.csv')
```

 Choose Files project_data.csv

- **project_data.csv**(text/csv) - 3671581 bytes, last modified: 5/18/2025 - 100% done



```
# drop Booking ID column since we said we won't be using it
data = data.drop(['Booking_ID'], axis=1)
```

```
# encode the target variable: booking_status
data['booking_status'] = data['booking_status'].map({'not_canceled': 0, 'canceled': 1})
```

```
# feature engineering steps
# take a deeper look at the time of the booking
data['arrival_date'] = pd.to_datetime(data['arrival_date'])
# break arrival up into year, month, day, and day of the week
data['arrival_year'] = data['arrival_date'].dt.year
data['arrival_month'] = data['arrival_date'].dt.month
data['arrival_day'] = data['arrival_date'].dt.day
data['arrival_dayofweek'] = data['arrival_date'].dt.dayofweek
# weekend arrival
data['is_weekend'] = data['arrival_dayofweek'].isin([5,6]).astype(int)
# week during the year arrival
data['arrival_weekofyear'] = data['arrival_date'].dt.isocalendar().week
# booking trend over time
reference_date = pd.to_datetime('2017-01-01')
data['days_since_reference'] = (data['arrival_date'] - reference_date).dt.days
# total nights stayed
data['total_nights'] = data['no_of_week_nights'] + data['no_of_weekend_nights']
# cancellation ratio
data['cancellation_ratio'] = data['no_of_previous_cancellations'] / (data['no_of_previous_bookings_not_canceled'] + 1)
```

```
# add seasonality into the mix
def get_season(month):
    if month in [12, 1, 2]:
        return 'winter'
    elif month in [3, 4, 5]:
        return 'spring'
    elif month in [6, 7, 8]:
        return 'summer'
    else:
        return 'fall'
```

```
data['season'] = data['arrival_month'].apply(get_season)
data = data.drop(['arrival_date'], axis=1) # can drop arrival_date now
```

```
# one-hot encode categorical features
categorical_cols = ['type_of_meal_plan', 'room_type_reserved', 'market_segment_type', 'season']
data = pd.get_dummies(data, columns=categorical_cols, drop_first=True)
```

```
# lets separate the features from the target
X = data.drop('booking_status', axis=1)
```

```

Y = data['booking_status']

# scale the features for a more comparable set of results
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# split into train and test, use scaled data, 20% test, 80% train
# enable reproducibility and prevent class imbalance using stratify
X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, Y, test_size=0.2, random_state=42, stratify=Y)

# convert data into tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
Y_train_tensor = torch.tensor(Y_train.values, dtype=torch.float32).unsqueeze(1)
Y_test_tensor = torch.tensor(Y_test.values, dtype=torch.float32).unsqueeze(1)

# build initial NN model
input_size = X_train.shape[1]
hidden_size = 64 # lets choose 64 to start for our hidden layer
# build model
model = nn.Sequential(
    nn.Linear(input_size, hidden_size),
    nn.ReLU(),
    nn.Linear(hidden_size, 1),
    nn.Sigmoid()
)

# use BCE Loss since we need binary classification
loss_fn = nn.BCELoss()
# use Adam optimizer for neural network
optimizer = optim.Adam(model.parameters(), lr=0.001)

# initial training loop for 50 epochs
epochs = 50
# create loop
for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()
    outputs = model(X_train_tensor)
    loss = loss_fn(outputs, Y_train_tensor)
    loss.backward()
    optimizer.step()
# design output setup
if (epoch+1) % 10 == 0:
    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

➡ Epoch 10, Loss: 0.6168
Epoch 20, Loss: 0.5836
Epoch 30, Loss: 0.5581
Epoch 40, Loss: 0.5373
Epoch 50, Loss: 0.5192

# we can now make predictions on test set
model.eval()
with torch.no_grad():
    Y_probs_original = model(X_test_tensor).numpy()
    Y_pred_original = (Y_probs_original >= 0.5).astype(int)

# calculate performance metrics
acc = accuracy_score(Y_test, Y_pred_original)
auc = roc_auc_score(Y_test, Y_probs_original)
print(f'Accuracy: {acc:.4f}, AUC: {auc:.4f}')

➡ Accuracy: 0.7601, AUC: 0.8103

# hyperparameter tuning
# lets use multiple hidden layers, learning rates, and epochs
hidden_sizes = [32, 64, 128]
learning_rates = [0.001, 0.0005]
epochs_list = [50, 100]

results = []
# create loop for each hyperparameter

```

```

for hidden_size in hidden_sizes:
    for lr in learning_rates:
        for num_epochs in epochs_list:
            print(f'Hidden: {hidden_size}, Learning rate: {lr}, Epochs: {num_epochs}')

            model = nn.Sequential(
                nn.Linear(input_size, hidden_size),
                nn.ReLU(),
                nn.Linear(hidden_size, 1),
                nn.Sigmoid()
            )

            loss_fn = nn.BCELoss()
            optimizer = optim.Adam(model.parameters(), lr=lr)

            for epoch in range(num_epochs):
                model.train()
                optimizer.zero_grad()
                outputs = model(X_train_tensor)
                loss = loss_fn(outputs, Y_train_tensor)
                loss.backward()
                optimizer.step()

            # lets evaluate the results
            model.eval()
            with torch.no_grad():
                Y_probs_original = model(X_test_tensor).numpy()
                Y_pred_original = (Y_probs_original >= 0.5).astype(int)
                acc = accuracy_score(Y_test, Y_pred_original)
                auc = roc_auc_score(Y_test, Y_probs_original)
                results.append((hidden_size, lr, num_epochs, acc, auc))

```

```

➡ Hidden: 32, Learning rate: 0.001, Epochs: 50
Hidden: 32, Learning rate: 0.001, Epochs: 100
Hidden: 32, Learning rate: 0.0005, Epochs: 50
Hidden: 32, Learning rate: 0.0005, Epochs: 100
Hidden: 64, Learning rate: 0.001, Epochs: 50
Hidden: 64, Learning rate: 0.001, Epochs: 100
Hidden: 64, Learning rate: 0.0005, Epochs: 50
Hidden: 64, Learning rate: 0.0005, Epochs: 100
Hidden: 128, Learning rate: 0.001, Epochs: 50
Hidden: 128, Learning rate: 0.001, Epochs: 100
Hidden: 128, Learning rate: 0.0005, Epochs: 50
Hidden: 128, Learning rate: 0.0005, Epochs: 100

```

```

# save model results to a dataframe
results_df = pd.DataFrame(results, columns=['Hidden', 'LR', 'Epochs', 'Accuracy', 'AUC'])
results_df = results_df.sort_values(by='AUC', ascending=False)
print(results_df)

```

```

➡

```

	Hidden	LR	Epochs	Accuracy	AUC
9	128	0.0010	100	0.760072	0.870838
5	64	0.0010	100	0.760072	0.849700
11	128	0.0005	100	0.760072	0.832360
8	128	0.0010	50	0.760072	0.829930
1	32	0.0010	100	0.760072	0.812962
4	64	0.0010	50	0.760072	0.810227
7	64	0.0005	100	0.760072	0.795502
0	32	0.0010	50	0.760072	0.794898
3	32	0.0005	100	0.760072	0.772592
10	128	0.0005	50	0.760072	0.769120
2	32	0.0005	50	0.760072	0.750883
6	64	0.0005	50	0.760072	0.731669

```

# for graphing purposes, lets use the model with best results
input_size = X_train.shape[1]
best_hidden = 128
best_lr = 0.001
best_epochs = 100
# build model
final_model = nn.Sequential(
    nn.Linear(input_size, best_hidden),
    nn.ReLU(),
    nn.Linear(best_hidden, 1),
    nn.Sigmoid()
)
# loss function
loss_fn = nn.BCELoss()
optimizer = optim.Adam(final_model.parameters(), lr=best_lr)

```

```

# train the final model, create loop
for epoch in range(best_epochs):
    final_model.train()
    optimizer.zero_grad()
    outputs = final_model(X_train_tensor)
    loss = loss_fn(outputs, Y_train_tensor)
    loss.backward()
    optimizer.step()

    if (epoch+1) % 10 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# evaluate the final model on the test data
final_model.eval()
with torch.no_grad():
    y_probs_original = final_model(X_test_tensor).numpy()
    y_pred_original = (y_probs_original >= 0.5).astype(int)

# print the final model metrics
acc = accuracy_score(Y_test, y_pred_original)
auc = roc_auc_score(Y_test, y_probs_original)
print(f'Final model -> Accuracy: {acc:.4f}, AUC: {auc:.4f}')

# plot the ROC curve, true positive versus false positive rate
fpr, tpr, thresholds = roc_curve(Y_test, y_probs_original)

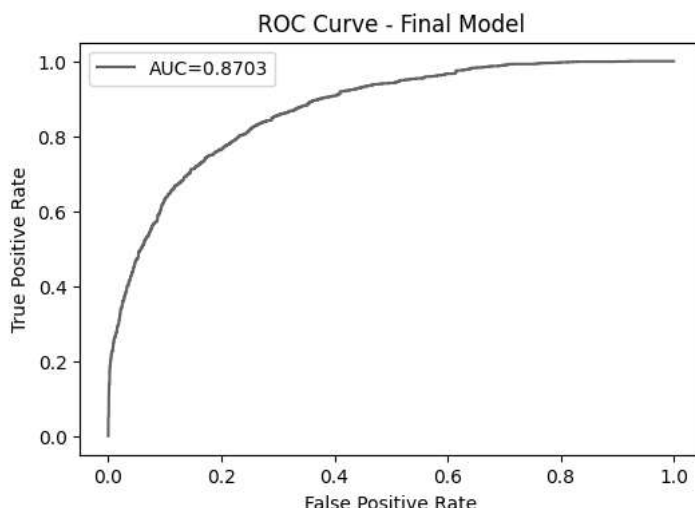
plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, label=f"AUC={auc:.4f}")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Final Model")
plt.legend()
plt.show()

```

```

Epoch 10, Loss: 0.6306
Epoch 20, Loss: 0.5811
Epoch 30, Loss: 0.5478
Epoch 40, Loss: 0.5213
Epoch 50, Loss: 0.4985
Epoch 60, Loss: 0.4785
Epoch 70, Loss: 0.4607
Epoch 80, Loss: 0.4451
Epoch 90, Loss: 0.4318
Epoch 100, Loss: 0.4206
Final model -> Accuracy: 0.8073, AUC: 0.8703

```

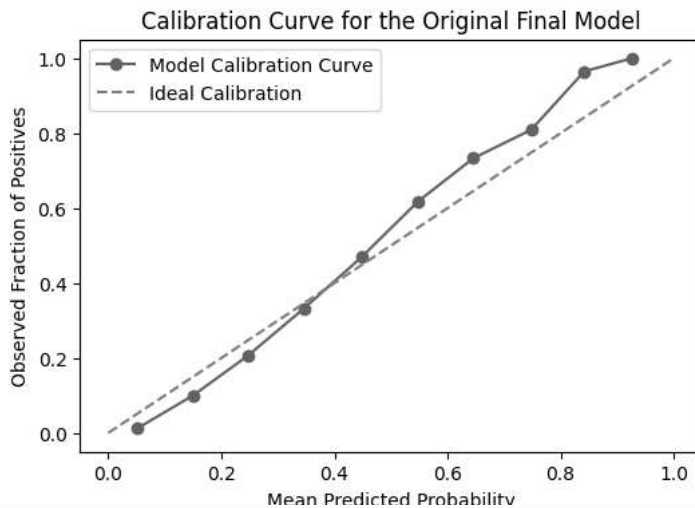


```

# import calibration curve library
from sklearn.calibration import calibration_curve
# build the calibration curve
prob_true, prob_pred = calibration_curve(Y_test, y_probs_original, n_bins=10)
plt.figure(figsize=(6, 4))
plt.plot(prob_pred, prob_true, marker='o', label='Model Calibration Curve')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Ideal Calibration')
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Observed Fraction of Positives')
plt.title('Calibration Curve for the Original Final Model')

```

```
plt.legend()
plt.show()
```



key new code I added for new modeling

```
# Create the new neural network with updated changes; trying to overfit
hidden_sizes = [128, 256] # add more neurons per layer
learning_rates = [0.001, 0.0005] # keep LR the same
epochs_list = [100, 200] # increase epoch counts to allow for overfitting
dropout_rates = [0.2, 0.5] # dropout rates to control overfitting
# store results
results = []

for hidden_size in hidden_sizes:
    for lr in learning_rates:
        for num_epochs in epochs_list:
            for dr in dropout_rates:
                print(f'Hidden: {hidden_size}, LR: {lr}, Epochs: {num_epochs}, Dropout: {dr}')
# use two hidden layers with more neurons and higher dropout
                model = nn.Sequential(
                    nn.Linear(input_size, hidden_size),
                    nn.ReLU(),
                    nn.Dropout(dr),
                    nn.Linear(hidden_size, hidden_size),
                    nn.ReLU(),
                    nn.Dropout(dr),
                    nn.Linear(hidden_size, 1),
                    nn.Sigmoid()
                )

                loss_fn = nn.BCELoss()
                optimizer = optim.Adam(model.parameters(), lr=lr)

                for epoch in range(num_epochs):
                    model.train()
                    optimizer.zero_grad()
                    outputs = model(X_train_tensor)
                    loss = loss_fn(outputs, Y_train_tensor)
                    loss.backward()
                    optimizer.step()

# evaluate model
                model.eval()
                with torch.no_grad():
                    Y_probs_new = model(X_test_tensor).numpy()
                    Y_pred_new = (Y_probs_new >= 0.5).astype(int)
                    acc1 = accuracy_score(Y_test, Y_pred_new)
                    auc1 = roc_auc_score(Y_test, Y_probs_new)
                    results.append((hidden_size, lr, num_epochs, dr, acc1, auc1))

# Summary of results, take best result
results_df = pd.DataFrame(results, columns=['Hidden', 'LR', 'Epochs', 'Dropout', 'Accuracy', 'AUC'])
results_df = results_df.sort_values(by='AUC', ascending=False)
print(results_df)
```

```

# Build the best model and evaluate results
best_row = results_df.iloc[0]
best_hidden = best_row['Hidden']
best_lr = best_row['LR']
best_epochs1 = int(best_row['Epochs'])
best_dropout = best_row['Dropout']
# print results
print(f"\nBest Model: Hidden={best_hidden}, LR={best_lr}, Epochs={best_epochs1}, Dropout={best_dropout}")
# build model
final_model = nn.Sequential(
    nn.Linear(input_size, int(best_hidden)),
    nn.ReLU(),
    nn.Dropout(best_dropout),
    nn.Linear(int(best_hidden), int(best_hidden)),
    nn.ReLU(),
    nn.Dropout(best_dropout),
    nn.Linear(int(best_hidden), 1),
    nn.Sigmoid()
)
# loss function
loss_fn = nn.BCELoss()
optimizer = optim.Adam(final_model.parameters(), lr=best_lr)

# train the final model, create loop
for epoch in range(best_epochs1):
    final_model.train()
    optimizer.zero_grad()
    outputs = final_model(X_train_tensor)
    loss = loss_fn(outputs, Y_train_tensor)
    loss.backward()
    optimizer.step()

    if (epoch+1) % 10 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# evaluate the final model on the test data
final_model.eval()
with torch.no_grad():
    y_probs_new = final_model(X_test_tensor).numpy()
    y_pred_new = (y_probs_new >= 0.5).astype(int)

# print the final model metrics
acc1 = accuracy_score(Y_test, y_pred_new)
auc1 = roc_auc_score(Y_test, y_probs_new)
print(f'Final overfitting-capable model -> Accuracy: {acc1:.4f}, AUC: {auc1:.4f}')

# plot the ROC curve, true positive versus false positive rate
fpr, tpr, thresholds = roc_curve(Y_test, y_probs_new)

plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, label=f"AUC={auc1:.4f}")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Overfitting-Capable Model")
plt.legend()
plt.show()

```

```

Hidden: 128, LR: 0.001, Epochs: 100, Dropout: 0.2
Hidden: 128, LR: 0.001, Epochs: 100, Dropout: 0.5
Hidden: 128, LR: 0.001, Epochs: 200, Dropout: 0.2
Hidden: 128, LR: 0.001, Epochs: 200, Dropout: 0.5
Hidden: 128, LR: 0.0005, Epochs: 100, Dropout: 0.2
Hidden: 128, LR: 0.0005, Epochs: 100, Dropout: 0.5
Hidden: 128, LR: 0.0005, Epochs: 200, Dropout: 0.2
Hidden: 128, LR: 0.0005, Epochs: 200, Dropout: 0.5
Hidden: 256, LR: 0.001, Epochs: 100, Dropout: 0.2
Hidden: 256, LR: 0.001, Epochs: 100, Dropout: 0.5
Hidden: 256, LR: 0.001, Epochs: 200, Dropout: 0.2
Hidden: 256, LR: 0.001, Epochs: 200, Dropout: 0.5
Hidden: 256, LR: 0.0005, Epochs: 100, Dropout: 0.2
Hidden: 256, LR: 0.0005, Epochs: 100, Dropout: 0.5
Hidden: 256, LR: 0.0005, Epochs: 200, Dropout: 0.2
Hidden: 256, LR: 0.0005, Epochs: 200, Dropout: 0.5

```

	Hidden	LR	Epochs	Dropout	Accuracy	AUC
10	256	0.0010	200	0.2	0.855546	0.923472
11	256	0.0010	200	0.5	0.846164	0.913831
2	128	0.0010	200	0.2	0.849476	0.913819
14	256	0.0005	200	0.2	0.842439	0.908867
8	256	0.0010	100	0.2	0.842577	0.905748
3	128	0.0010	200	0.5	0.840922	0.903030
15	256	0.0005	200	0.5	0.838024	0.901131
6	128	0.0005	200	0.2	0.833609	0.899632
9	256	0.0010	100	0.5	0.833609	0.898974
0	128	0.0010	100	0.2	0.827677	0.895172
7	128	0.0005	200	0.5	0.829194	0.892230
12	256	0.0005	100	0.2	0.826297	0.891610
13	256	0.0005	100	0.5	0.820502	0.888272
1	128	0.0010	100	0.5	0.820226	0.888199
4	128	0.0005	100	0.2	0.816777	0.881333
5	128	0.0005	100	0.5	0.810292	0.874276

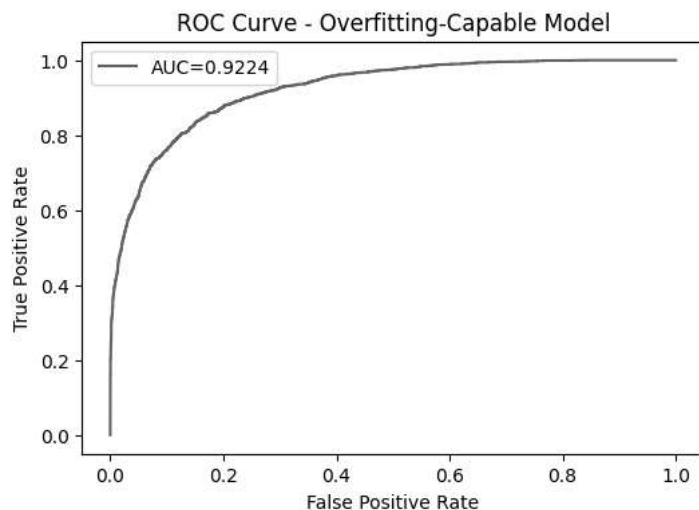
Best Model: Hidden=256.0, LR=0.001, Epochs=200, Dropout=0.2

```

Epoch 10, Loss: 0.5463
Epoch 20, Loss: 0.4639
Epoch 30, Loss: 0.4234
Epoch 40, Loss: 0.4096
Epoch 50, Loss: 0.3972
Epoch 60, Loss: 0.3898
Epoch 70, Loss: 0.3824
Epoch 80, Loss: 0.3739
Epoch 90, Loss: 0.3680
Epoch 100, Loss: 0.3626
Epoch 110, Loss: 0.3595
Epoch 120, Loss: 0.3550
Epoch 130, Loss: 0.3507
Epoch 140, Loss: 0.3470
Epoch 150, Loss: 0.3427
Epoch 160, Loss: 0.3393
Epoch 170, Loss: 0.3361
Epoch 180, Loss: 0.3334
Epoch 190, Loss: 0.3296
Epoch 200, Loss: 0.3262

```

Final overfitting-capable model -> Accuracy: 0.8560, AUC: 0.9224

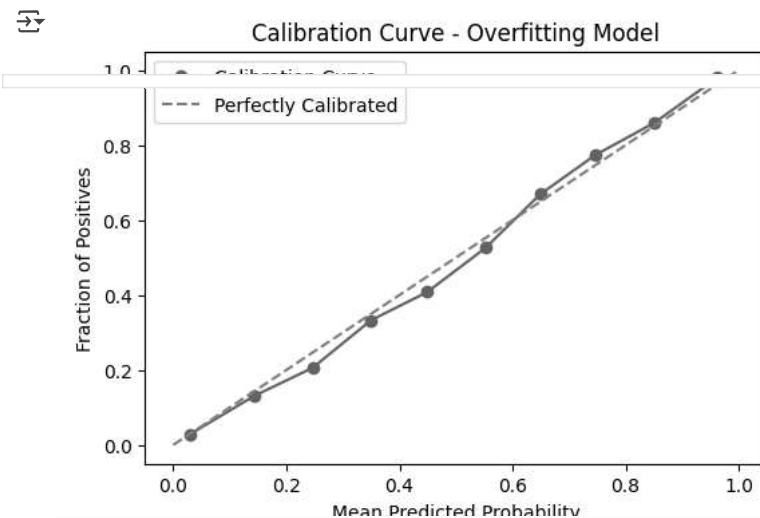


```

# build the calibration curve
prob_true, prob_pred = calibration_curve(Y_test, y_probs_new, n_bins=10)
plt.figure(figsize=(6, 4))

```

```
plt.plot(prob_pred, prob_true, marker='o', label='Calibration Curve')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Perfectly Calibrated')
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
plt.title('Calibration Curve - Overfitting Model')
plt.legend()
plt.show()
```



```
# calculate the correlation matrix
correlation = data.corr()
# display the correlations of the variables with booking_status as target
correlation_target = correlation['booking_status'].sort_values(ascending=False)
print(correlation_target)
```

```
booking_status      1.000000
lead_time           0.438629
arrival_year         0.179762
days_since_reference 0.172698
avg_price_per_room   0.142721
season_summer         0.110196
market_segment_type_online 0.106131
total_nights         0.103474
no_of_week_nights    0.092904
no_of_adults         0.086671
type_of_meal_plan_meal_plan_2 0.086550
no_of_weekend_nights 0.061704
no_of_children        0.033033
room_type_reserved_room_type6 0.032888
arrival_dayofweek     0.030317
is_weekend           0.029501
season_spring         0.017502
room_type_reserved_room_type4 0.013443
arrival_day           0.011109
type_of_meal_plan_not_selected 0.002838
room_type_reserved_room_type2 0.000506
room_type_reserved_room_type3 -0.001246
type_of_meal_plan_meal_plan_3 -0.003198
room_type_reserved_room_type5 -0.009839
arrival_weekofyear    -0.010569
arrival_month         -0.011789
room_type_reserved_room_type7 -0.014088
cancellation_ratio    -0.019806
no_of_previous_cancellations -0.033871
market_segment_type_offline -0.038283
no_of_previous_bookings_not_canceled -0.060046
market_segment_type_complementary -0.072834
required_car_parking_space -0.086053
repeated_guest        -0.107490
market_segment_type_corporate -0.112768
season_winter         -0.164675
no_of_special_requests -0.253339
Name: booking_status, dtype: float64
```